

Agentic AI: Foundations and Open-Source Practice

Day 3 Study Material

Building an Agent from Scratch, and How Agents Fail

1. Session Overview

Unit	Unit 1: Foundations of AI Agents
Sub-topics	1.5 Building an agent from scratch: agent loop, tool registry, JSON tool schemas, parsing tool calls, stopping conditions, max iterations 1.6 Failure modes: infinite loops, hallucinated tool calls, context overflow, cost growth
Course Outcome	CO1
Builds on	Day 1: the agent you ran. Day 2: the five components and the ReAct pattern.

Learning Objectives

By the end of Day 3, you will be able to:

1. Write an agent loop from scratch and explain every line of it.
2. Build a tool registry and add a new tool to an agent in three steps.
3. Write a JSON tool schema and explain each field.
4. Parse and validate tool calls safely, including malformed and unknown ones.
5. List the stopping conditions an agent needs and choose a sensible iteration limit.
6. Recognise the four main failure modes and explain what causes each one.
7. Apply guards that prevent infinite loops, context overflow, and runaway cost.

2. Video Lectures

Watch Video 1 before class. The others give alternative walkthroughs of the same build.

No.	Title and Link	When and Why
1	Build an AI Agent From Scratch With Python https://www.youtube.com/watch?v=jcpZU2vyrf4	Watch before class. Builds an agent from the model, tools and system prompt, exactly as we do today.
2	Intro to Agents: Create an Agent from Scratch (No Frameworks) https://www.youtube.com/watch?v=vHDwpoSFdQY	Recommended. A no-framework build using Hugging Face inference, close to our Option C setup.
3	Build an AI Agent From Scratch in Python: Tutorial for Beginners https://www.youtube.com/watch?v=bTMPwUgLZf0	Optional. A slower walkthrough for anyone who found the code hard to follow.
4	ReAct AI Agents, clearly explained! https://www.youtube.com/watch?v=vFdlrZyKEwQ	Revision of Day 2. The loop you build today is a ReAct loop.

Videos use different models and providers from ours. The ideas are the same; follow the code in your lab manual for the exact commands.

3. Lecture Notes: 1.5 Building an Agent from Scratch

Frameworks such as LangChain and CrewAI hide the agent loop behind a single function call. That is convenient, but when an agent misbehaves you cannot debug what you cannot see. Today you build the loop yourself, so that every later framework becomes a shortcut you understand rather than magic you trust.

3.1 The Six Pieces of a Hand-Built Agent

Piece	What it does	Where it lives
System prompt	States the role, the rules, and the tool policy	A string constant
Message list	Holds the whole task history: system, user, assistant, tool	A Python list of dictionaries
Tool registry	Maps a tool name to the real Python function	A dictionary: name to function
Tool schemas	Describes each tool to the model in JSON Schema	A list of dictionaries sent with every call
The loop	Calls the model, runs tools, feeds results back	A for loop with a step limit
Stopping conditions	Decide when the run ends, normally or otherwise	Checks inside the loop

3.2 The Agent Loop

```
messages = [ {system prompt}, {user question} ]

for step in 1 .. max_steps:
    reply = LLM(messages, tools=schemas)           # 1. REASON

    if reply has no tool calls:                       # 2. STOP
        return reply.text

    messages.append(reply)                           # 3. RECORD the request

    for call in reply.tool_calls:                     # 4. ACT (may be several)
        arguments = parse(call.arguments)
        result     = registry[call.name](**arguments)
        messages.append({role: "tool", id: call.id, content: result}) # 5. OBSERVE

return "stopped: step limit reached"                # 6. SAFETY EXIT
```

Step	Why it is written this way
1. Reason	The model sees the entire history every time, because the API is stateless.
2. Stop	The model signals completion by replying with text instead of a tool call. Your code does not decide when the task is done.
3. Record	The assistant's tool request must be stored before the results, or the results have nothing to attach to and the API rejects the next call.

Step	Why it is written this way
4. Act	A model can request several tools in one turn (parallel tool calls), so this is a loop, not a single call.
5. Observe	Each result is added with role "tool" and the id of the request that produced it.
6. Safety exit	If the loop finishes without the model stopping, the run is abandoned deliberately rather than continuing forever.

3.3 The Tool Registry

The registry is a dictionary that maps the name the model uses to the function your program runs.

```

TOOL_FUNCTIONS = {
    "get_course_fee": get_course_fee,
    "calculator": calculator,
}

# Looking a tool up, safely
function = TOOL_FUNCTIONS.get(name)
result = function(**arguments) if function else f"Unknown tool: {name}"

```

Point	Why it matters
A dictionary, not if/else	Adding a tool means adding one line, not editing the loop. The loop never changes as the agent grows.
.get() instead of []	A model can invent a tool name. .get() returns None, so you can reply with a message instead of crashing.
Names must match the schema exactly	The schema name is what the model sends back; the registry key is how you find the function. A mismatch is a common bug.
Adding a tool takes three steps	Write the function, add it to the registry, add its schema. Nothing else changes.

3.4 JSON Tool Schemas

The model cannot see your code. All it gets is this description, sent with every request.

```

{
  "type": "function",
  "function": {
    "name": "get_course_fee",
    "description": "Get the fee in rupees for a single course code, for example CS101.",
    "parameters": {
      "type": "object",
      "properties": {
        "course_code": {"type": "string", "description": "Course code such as CS101"}
      },
      "required": ["course_code"]
    }
  }
}

```

Field	Notes
name	Must match the registry key exactly. Use a clear verb-like name: get_course_fee, not tool1.

Field	Notes
description	The single most important field. This is prompt text. Say what the tool does, what it returns, and when to use it, and give an example value. Vague descriptions cause wrong tool choices.
parameters.type	Always "object": arguments are sent as a JSON object even when there is only one.
properties	One entry per argument, each with a type and ideally its own description. Types: string, number, integer, boolean, array, object.
required	Arguments the model must supply. Without this, small models sometimes send an empty object.
enum (optional)	Restricts a value to a fixed list, for example "semester": {"enum": ["odd", "even"]}. Very effective at stopping invented values.

A useful habit: read your schema aloud and ask whether a new student could choose the right tool from that sentence alone. If not, the model cannot either.

3.5 Parsing Tool Calls

The model's request arrives as structured data, but the **arguments are a JSON string**, not a dictionary, and the model wrote that string. Anything can be in it.

```
arguments = json.loads(call.function.arguments or "{}")    # may raise
result = registry[call.name](**arguments)                # may raise
```

What can go wrong	Defence
Malformed JSON	Wrap json.loads in try/except and return the error text to the model so it can retry.
Empty arguments string	Use `or "{}"` so an empty string parses as an empty object.
Unknown tool name	Look the name up with .get() and return "Unknown tool: ..." instead of raising.
Wrong or extra argument names	function(**arguments) raises TypeError. Catch it and return the message; the model usually corrects itself on the next step.
Wrong type ("12000" instead of 12000)	Convert inside the tool, or validate before calling. Tools should be forgiving, because the caller is a language model.
Dangerous content in an argument	Never pass an argument into eval, a shell command, or an unchecked SQL query. This is the Unit 8 security topic in miniature.

The rule: a tool should return an error message, never raise an exception out of the loop. An error the model can read is a chance to recover; a crash ends the run.

3.6 Stopping Conditions

An agent needs more than one way to stop. A real system usually has all of these:

Stopping condition	When it fires
No tool call in the reply	The normal, successful ending: the model has answered.
Maximum iterations reached	The safety net. The run is abandoned and reported.

Stopping condition	When it fires
Token or cost budget exceeded	Stops an expensive run even if the step count is low.
Wall-clock timeout	Protects a user waiting on a response.
Repeated identical tool call	A loop detector: the same call with the same arguments twice or three times in a row means no progress.
Too many tool errors	After a few failures in a row, stopping and reporting beats blind retrying.
An explicit finish tool	Some designs give the model a <code>final_answer</code> tool; the run ends when it is called. This makes stopping explicit rather than implied.
Human interrupt	A person cancels, or declines an approval request.

3.7 Choosing `max_iterations`

The limit should be a little above the longest sensible task, and no higher.

If the limit is...	What happens
Too low (1 or 2)	Correct answers get cut off mid-task. Users see "step limit reached" on questions the agent could have solved.
Well chosen (about 2x the typical task)	Normal tasks finish; broken runs stop quickly.
Too high (50 or more)	A confused agent burns minutes and money before anyone notices, and the message list may overflow the context window first.

Practical method: run your real questions, note the highest step count, and set the limit to roughly double it. Log every run that hits the limit, because those logs tell you whether the limit is wrong or the agent is.

4. Lecture Notes: 1.6 Failure Modes

Ordinary software fails by crashing. Agents usually fail while still running and still sounding confident, which is why guards must be designed in from the start.

4.1 Infinite Loops

Aspect	Detail
What it looks like	The same tool called again and again with the same arguments, or two tools alternating forever, until the step limit ends the run.
Why it happens	The observation does not answer the model's question; the tool keeps returning an error; the system prompt demands something impossible; or a small model simply does not notice it already has the answer.
How to detect it	Keep a record of (tool name, arguments) pairs and count repeats. Two identical calls in a row is a warning; three is a loop.
How to prevent it	A maximum step limit, a repeat detector that breaks the loop, clearer tool descriptions, and returning specific error messages rather than the same unhelpful one.

4.2 Hallucinated Tool Calls

The model invents something about the tools instead of using them correctly. Four varieties:

Variety	Example and defence
Invented tool name	Calls <code>send_email</code> when no such tool exists. Defence: registry lookup with <code>.get()</code> , return "Unknown tool".
Invented arguments	Calls <code>get_course_fee("ME404")</code> for a course that does not exist. Defence: the tool returns "Unknown course code", and the system prompt lists valid codes.
Wrong tool for the job	Uses the calculator to look up a fee. Defence: sharper tool descriptions, and a firmer tool policy in the system prompt.
Fabricated result	Skips the tool entirely and states a fee from imagination. Defence: "Never guess: always use the tool" in the prompt, and for critical systems, checking that an answer is supported by a tool result before showing it.

Small models hallucinate tool calls far more than large ones. If your agent misbehaves in the lab, note whether a bigger model fixes it: model choice is part of agent engineering.

4.3 Context Overflow

Every step adds messages, and the whole list is resent on every call. Eventually the conversation no longer fits in the model's context window.

Aspect	Detail
Symptoms	An error about context length, silently truncated input, the agent "forgetting" the original question, or answers that ignore earlier tool results.
Main cause	Large tool outputs. One document search or database query can return thousands of tokens, and it is repeated in every later call.
Defence 1: truncate tool output	Return the first N characters, or a summary, plus a way to fetch more if needed.
Defence 2: trim old messages	Keep the system prompt, the original question, and the most recent exchanges; drop the middle.
Defence 3: summarize	Replace older turns with a short summary written by the model.
Defence 4: store outside	Keep bulky results in a file or database and pass only a reference into the conversation.

4.4 Cost and Latency Growth

An agent is not one LLM call. It is one call per step, and each call carries the entire history.

```
step 1:  system + question                = 200 tokens
step 2:  ... + tool call + result          = 400 tokens
step 3:  ... + tool call + result          = 700 tokens
step 4:  ... + tool call + result          = 1100 tokens
                    total sent = 2400 tokens
                    for a task a human would call "one question"
```

Driver of cost	What to do about it
Number of steps	Fewer, better tools; clearer prompts; Plan-and-Execute for long tasks.
History resent every step	Trim and summarize; keep tool outputs small.
Model size	Use a small model for routing and simple steps, a large one only where it is needed.
Retries after failures	Fix the cause. Every retry is a full extra call.
Latency	Parallel tool calls where possible, caching of repeated lookups, and streaming so the user sees progress.
No budget	Set a maximum token or rupee budget per run, and stop when it is crossed.

A useful number to remember: if each step is roughly twice the size of the last, doubling the step limit can more than double the cost. Limits protect budgets, not just correctness.

4.5 Other Failure Modes Worth Knowing

Failure	Description
Premature stopping	The agent answers before finishing the task, often because the prompt did not state what a complete answer contains.
Error cascade	An early wrong observation is treated as fact and poisons every later step.
Non-determinism	The same question takes different paths on different runs, which makes bugs hard to reproduce. Use temperature 0 and log everything.
Silent tool failure	A tool returns an empty result rather than an error, and the agent reports confidently on nothing.
Over-tooling	Twenty tools where five would do, so the model chooses badly.
Prompt injection (Unit 8)	Text inside a document or web page instructs the agent to do something else. A serious failure mode once tools can write.

4.6 A Defensive Checklist

Before any agent you write goes near real users, check that:

1. It has a maximum step limit, and the limit is logged when it is hit.
2. Every tool returns a readable error message instead of raising an exception.
3. Unknown tool names and malformed arguments are handled without crashing.
4. Tool outputs are bounded in size.
5. Repeated identical tool calls are detected and stopped.
6. There is a token, time, or cost budget for a single run.
7. Every step is logged: the tool, the arguments, the result, and the token count.
8. Any tool that changes data is read-only, restricted, or requires human approval.

5. Worked Example: A Run That Goes Wrong

A student asks: "What is the fee for ME404 with a 20% scholarship?" ME404 does not exist. Here is how a well-guarded agent behaves, and which guard acts at each point.

Step	What happens	Guard at work
1	The model calls <code>get_course_fee("ME404")</code> .	(normal tool call)
2	The tool returns "Unknown course code: ME404" instead of raising <code>KeyError</code> .	Tools return errors, never crash
3	The model tries <code>get_course_fee("ME404")</code> again, unchanged.	—
4	The repeat detector notices the identical call and breaks the loop.	Repeat detection
5	The agent replies that ME404 is not in the fee list, and names the valid codes.	Clear error text plus a helpful prompt

Without those guards, the same run would have crashed at step 2 with a `KeyError`, or spun until the step limit at step 4, or confidently invented a fee at step 5. Each guard is small; together they are the difference between a demo and a system.

6. Key Terms

Term	Meaning
Agent loop	The repeated cycle of calling the model, running tools, and feeding results back
Tool registry	A dictionary mapping tool names to the functions that implement them
JSON Schema	The structured description of a tool's name, purpose, and parameters
Tool call	The model's structured request containing a tool name and JSON arguments
Parsing	Converting the model's argument string into a Python object safely
Stopping condition	Any rule that ends a run, normally or defensively
Max iterations	The hard limit on how many steps one run may take
Repeat detection	Spotting identical tool calls in succession to catch loops
Context overflow	The conversation growing beyond the model's context window
Truncation	Cutting a long tool output down to a safe size
Token budget	A limit on tokens spent in a single run
Error cascade	An early wrong observation corrupting all later steps
Graceful degradation	Failing with a clear, useful message instead of crashing or inventing

7. Further Reading

1. Anthropic, "Building effective agents" (loops, tools, and when simpler patterns are better) <https://www.anthropic.com/engineering/building-effective-agents>
2. S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR 2023 <https://arxiv.org/abs/2210.03629>
3. JSON Schema, the specification used for tool parameters

<https://json-schema.org/understanding-json-schema/>

4. T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," NeurIPS 2023
<https://arxiv.org/abs/2302.04761>
5. OWASP Top 10 for LLM Applications (excessive agency and unbounded consumption)
<https://genai.owasp.org>

Text book reading: Chip Huyen, AI Engineering (O'Reilly, 2025), the chapter on RAG and agents, sections on tool use, planning, and agent failure modes.